```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# ------------------------------------
# Load Dataset
# ------------------------------------
df = pd.read_csv("Iris.csv")

print("Dataset Shape:", df.shape)

print("\nFirst 5 Rows:")
print(df.head())

print("\nDataset Information:")
print(df.info())

print("\nMissing Values:")
print(df.isnull().sum())

# ------------------------------------
# Remove Id Column (if present)
# ------------------------------------
if 'Id' in df.columns:
    df = df.drop('Id', axis=1)

# ------------------------------------
# Features and Target
# ------------------------------------
X = df.iloc[:, :-1]
y = df.iloc[:, -1]

# ------------------------------------
# Train-Test Split
# ------------------------------------
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# ------------------------------------
# Feature Scaling
# ------------------------------------
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# ------------------------------------
# Finding Best K
# ------------------------------------
accuracy_list = []

for k in range(1, 21):
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)

    pred = knn.predict(X_test)

    accuracy_list.append(
        accuracy_score(y_test, pred)
    )

# Plot Accuracy vs K
plt.figure(figsize=(8,5))
plt.plot(range(1,21), accuracy_list, marker='o')
plt.xlabel("K Value")
plt.ylabel("Accuracy")
plt.title("Choosing Best K")
plt.grid()
plt.show()
```

```
# ------------------------------------
# Train Final Model
# ------------------------------------
knn = KNeighborsClassifier(n_neighbors=5)

knn.fit(X_train, y_train)

y_pred = knn.predict(X_test)

# ------------------------------------
# Accuracy
# ------------------------------------
print("\nAccuracy Score:")
print(accuracy_score(y_test, y_pred))

# ------------------------------------
# Confusion Matrix
# ------------------------------------
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6,4))
sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues'
)

plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

# ------------------------------------
# Classification Report
# ------------------------------------
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# ------------------------------------
# Pairplot Visualization
# ------------------------------------
sns.pairplot(
    df,
    hue=df.columns[-1]
)

plt.show()

# ------------------------------------
# Insights
# ------------------------------------
print("\nInsights:")
print("1. KNN classifies samples based on nearest neighbors.")
print("2. Feature scaling is important because KNN uses distance calculations.")
print("3. Different K values produce different accuracies.")
print("4. Very small K may cause overfitting.")
print("5. Very large K may cause underfitting.")
print("6. Iris dataset is well suited for KNN classification.")
```